Operation	Time Complexity	Auxiliary Space	Analysis / Justification
(i) Maximum Element	O(n)	O(1)	Single linear scan comparing each element to the current max. Req
(ii) 1st & 2nd Largest	O(n)	O(1)	Single scan updating max1 and max2. Minimum comparisons need
(iii) Mean	O(n)	O(1)	Requires a single pass to sum all elements, followed by a division b
(iv) Median	O(n)	O(1) or O(n)	Using the Median-of-Medians (Quickselect) algorithm guarantees d
(v) Standard Deviation	O(n)	O(1)	Computable in 2 linear passes (one for mean, one for variance) or 1
(vi) Mode	O(n2) array / O(nlogn) sorted / O(n) hash	O(1) to O(n)	Standard array in-place without hash maps requires sorting first (O(
(vii) Remove Duplicates	O(n2) in-place / O(nlogn) sorted	O(1) to O(n)	Without extra space, nested loops yield O(n2) worst-case. Sorting fi
(viii) Reverse Array	O(n)	O(1)	Two-pointer technique swapping outer pairs moving towards the ce
(ix) Partition around Pivot	O(n)	O(1)	Two-pointer in-place partition scan (Lomuto or Hoare variant config

Operation	Time Complexity	Auxiliary Space	Analysis / Justification
(i) Maximum Element	O(n)	O(1)	Single linear scan comparing each element to the current max. Requires n−1 comparisons.
(ii) 1st & 2nd Largest	O(n)	O(1)	Single scan updating max1 and max2. Minimum comparisons needed is n+⌈log2n⌉−2 (tournament method), but both are O(n).
(iii) Mean	O(n)	O(1)	Requires a single pass to sum all elements, followed by a division by n.
(iv) Median	O(n)	O(1) or O(n)	Using the Median-of-Medians (Quickselect) algorithm guarantees deterministic worst-case time of O(n). Sorting takes O(nlogn).
(v) Standard Deviation	O(n)	O(1)	Computable in 2 linear passes (one for mean, one for variance) or 1 pass using Welford's algorithm.
(vi) Mode	O(n2) array / O(nlogn) sorted / O(n) hash	O(1) to O(n)	Standard array in-place without hash maps requires sorting first (O(nlogn)) or frequency counting (O(n2) time, O(1) space). Hash t
(vii) Remove Duplicates	O(n2) in-place / O(nlogn) sorted	O(1) to O(n)	Without extra space, nested loops yield O(n2) worst-case. Sorting first allows in-place deduplication in O(nlogn) total time.
(viii) Reverse Array	O(n)	O(1)	Two-pointer technique swapping outer pairs moving towards the center (n/2 swaps).
(ix) Partition around Pivot	O(n)	O(1)	Two-pointer in-place partition scan (Lomuto or Hoare variant configured for ≥ and < conditions).

Operation	Time Complexity	Auxiliary Space	Justification / Method
(i) Matrix Additio	O(n2)	O(n2)	Must iterate over all n2 entries and add corresponding elements.
(ii) Matrix Multipl	O(n3) standardC	O(1) or O(n2)	Standard 3-nested loops yield n3 scalar multiplications. Theoretical lower bound is O(nω) (ω≈2.37).
(iii) Zero Matrix (	O(n2)	O(1)	Worst-case requires inspecting all n2 elements until a non-zero element or end is reached.
(iv) Symmetric M	O(n2)	O(1)	Checks A[i][j]==A[j][i] for i<j, requiring 2n(n−1) comparisons.
(v) Determinant	O(n3)	O(1) or O(n2)	Gaussian elimination / LU Decomposition reduces matrix to upper triangular in O(n3); determinant is the product of diagonal elements.
(vi) In-Situ Trans	O(n2)	O(1)	Swaps upper triangle elements A[i][j] with lower triangle A[j][i] in-place.
(vii) Eigenvalues	O(n3) per iteratic	O(n2)	Numerical methods like Power Iteration (dominant eigenvalue, O(n2) per step) or QR Algorithm (all eigenvalues, O(n3) total). Exact alg

Part 1: Sorting in $\mathcal{O}(n)$ Reversals (Mathematical Proof)

Theorem: Any permutation p of $\{1,2,\ldots,n\}$ can be sorted using at most $2n-3$ reversals, which is $\mathcal{O}(n)$ operations.

Proof Strategy (Induction/Constructive Elimination):

1. **Base Case:** For $n=1$ or $n=2$, at most 1 reversal is needed to sort the sequence.
2. **Induction Step:** Assume any permutation of size $k<n$ can be placed in its relative positions. To place element i into index i (where $1 \le i \le n$):
    - Locate the current position of the integer i, say at index k ($i \le k \le n$).
    - **Case 1 ($k=i$):** Element i is already in its correct position. 0 reversals needed.
    - **Case 2 ($k>i$):** Apply the operation reverse(p,i,k). This brings the integer i from position k directly to position i.
3. **Upper Bound:** Since each element i requires at most 1 reversal to be placed at index i, placing all n elements in order requires at most $n-1$ total reversals.

Thus, every permutation can be sorted using $\le n-1$ reversals, proving the $\mathcal{O}(n)$ upper bound on the number of reversals.

Part 2: Sorting with $\mathcal{O}(n \log^2 n)$ Total Cost Algorithm

When the cost of reverse(p, i, j) is bounded by its subsegment length $L = j - i + 1$, a simple selection approach takes $\mathcal{O}(n^2)$ total cost. To achieve $\mathcal{O}(n \log^2 n)$ total reversal cost, we use a **Divide-and-Conquer (Merge-Sort variant) approach**.

Algorithm Design

Instead of placing elements one by one into their final absolute positions, we sort sub-ranges recursively:

1. **Base Case:** A subarray of size 1 is already sorted (cost 0).
2. **Divide:** Split the array range $[L, R]$ into two equal halves: left $[L, M]$ and right $[M + 1, R]$. Recursively sort both halves.
3. **Conquer / Merge Step:**
    - After the recursive step, the left half contains elements that belong to the combined range, but they need to be partitioned so that all elements belonging to the left sub-range come before those belonging to the right sub-range.
    - By identifying contiguous blocks in the middle that are "out of place" (elements in the left half destined for the right side, and elements in the right half destined for the left side), we can apply a **single block-reversal** (a standard block-swap technique via 3 reversals) to place the blocks into their respective halves.
    - Applying this median-based division recursively brings the total cost down to $\mathcal{O}(n \log^2 n)$.

Cost & Time Complexity Analysis

- **Merge Cost at Level k:** At recursion depth d, merging two sorted segments of length $K = 2^d$ requires bounded reversals whose total span is at most $2K$.
- **Recurrence Relation for Cost $C(n)$:**

$$C(n) = 2C\left(\frac{n}{2}\right) + \mathcal{O}(n \log n)$$

Solving via the Master Theorem yields a total reversal cost of:

$$\text{Cost} = \mathcal{O}(\mathbf{n \log^2 n})$$